DAY 26/55

for loop

Loop through collections

Cleaner iteration

```
data = [10, 20, 30]
for item in data:
    print(item)
```

for loops visit each value in a sequence.

Less counter management. More readable code.

DAY 26/55

Python keeps loops simple

while

```
i = 0
while i < len(data):
    print(data[i])
    i += 1
```

for

```
for value in data:
    print(value)
```

Python gives you two main loops: while and for.

DAY 26/55

while needs manual control

Manual iteration

```
data = [10, 20, 30, 40]
i = 0                    # initialize
while i < len(data):     # condition
    print(data[i])
    i += 1               # increment
```

Initialize

Create an index before the loop.

Condition

Keep checking the index boundary.

Increment

Update it or risk an infinite loop.

DAY 26/55

for does the walking

Collection iteration

```
data = [10, 20, 30, 40]
for value in data:
    print(value)
```

No index

Python gives you one item at a time.

No len()

You do not need to compare positions.

No i += 1

The loop advances automatically.

DAY 26/55

Need numbers?
Use range()

Repeat 5 times

```
for i in range(5):
    print("Hello")

# i becomes:
0, 1, 2, 3, 4
```

Custom range

```
for n in range(1, 6):
    print(n)

# output:
1 2 3 4 5
```

range() creates a sequence when you do not already have one.

DAY 26/55

Cheat Sheet

Python for loops

Basic syntax

```
for item in collection:
    use(item)

for i in range(5):
    repeat_work()
```

Use for

Lists, strings, tuples, and other sequences.

Cleaner than while

No manual index setup for collections.

Use range()

Repeat a fixed number of times.

Still use while

When repetition depends on a changing condition.

Save this for your Python journey.

Day 26/55 - Prepared by Satyanarayan Sen